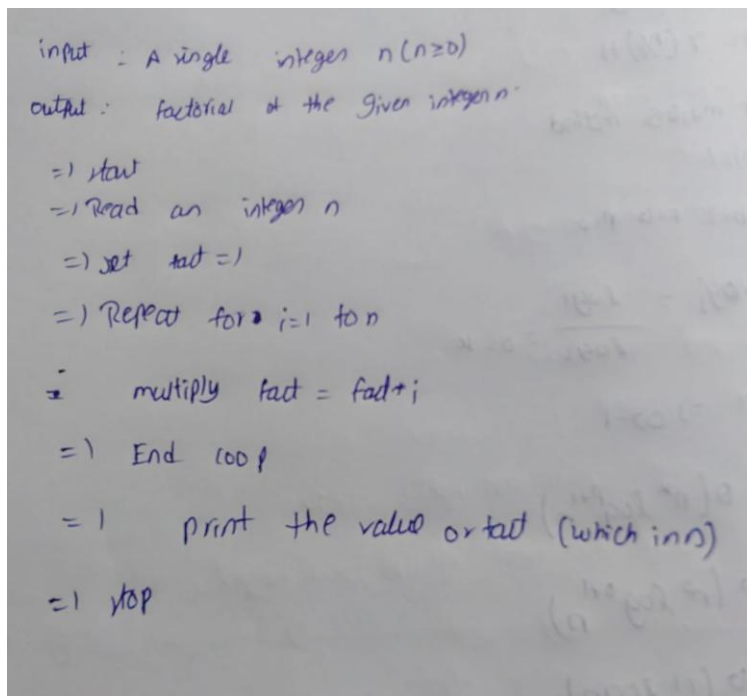


PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

FACTORIAL BY ITERATIVE

ALGORITHM:



input : A single integer $n (n \geq 0)$
output : factorial of the given integer n

- => start
- => Read an integer n
- => set $fact = 1$
- => Repeat for $i = 1$ to n
 - = multiply $fact = fact * i$
- => End loop
- => print the value of $fact$ (which is $n!$)
- => stop

PROGRAM:

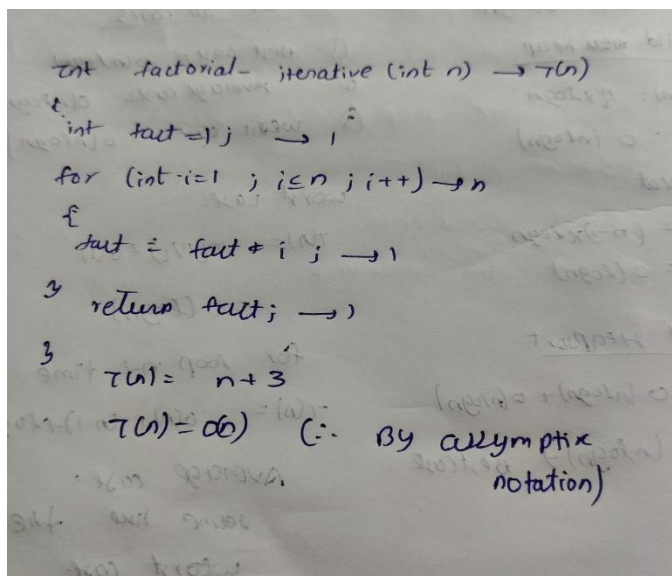
```
#include <iostream>
using namespace std;
int factorial_iterative(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}
```

```
}  
  
int main() {  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
    cout << "Factorial (Iterative): " << factorial_iterative(n) << endl;  
    return 0;  
}
```

OUTPUT:

```
Enter a number: 5  
Factorial (Iterative): 120
```

TIME COMPLEXITY:



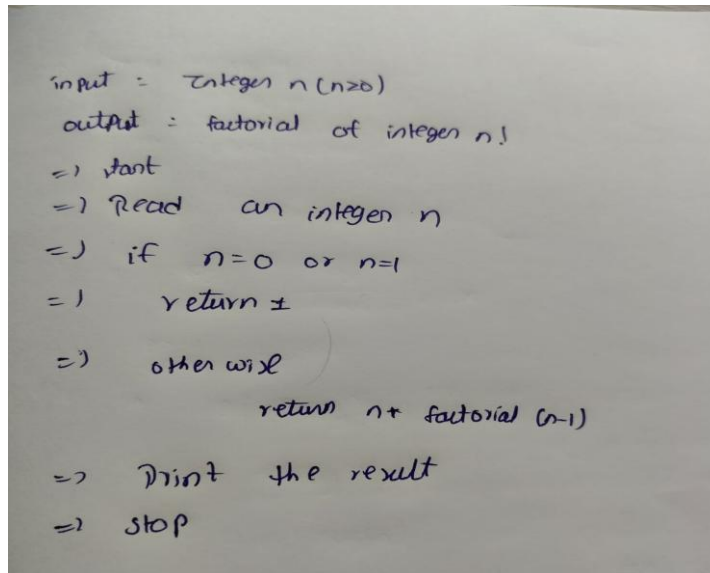
Handwritten analysis of the iterative factorial function's time complexity:

```
int factorial_iterative(int n) → T(n)  
{  
    int fact = 1; j → 1  
    for (int i = 1; i ≤ n; i++) → n  
    {  
        fact = fact * i; → 1  
    }  
    return fact; → 1  
}
```

∴ $T(n) = n + 3$
 $T(n) = O(n)$ (∴ By asymptotic notation)

FACTORIAL BY RECURSIVE

ALGORITHM:



```
input = Integer n (n > 0)
output = factorial of integer n!
=> start
=> Read an integer n
=> if n = 0 or n = 1
=> return 1
=> otherwise
    return n * factorial (n-1)
=> Print the result
=> stop
```

PROGRAM:

```
#include <iostream>

using namespace std;

int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Recursive): " << factorial_recursive(n) << endl;
    return 0;
}
```

OUTPUT:

```
Enter a number: 5
Factorial (Recursive): 120
```

TIME COMPLEXITY:

```
int factorial-recursive (int n) → T(n)
{
    if (n==0 || n==1)
        return 1;
    else
        return n * factorial-recursive (n-1)
}
↳ T(n) = T(n-1) + O(1)
= T(n-1) + 1
```

```
n → n-1 put in ①
T(n-1) = T(n-1) + 1 → ①
substitute ① in ①
T(n) = T(n-1) + 1
T(n) = T(n-2) + 2 → ②
n → n-2 put in ②
T(n-2) = T(n-2) + 2
= T(n-4) + 2 → ③
substitute ③ in ②
T(n) = T(n-4) + 2 + 2
T(n) = T(n-4) + 4
n → n-4
T(n-4) = T(n-8) + 4
substitute
T(n) = T(n-8) + 8 (i.e. n → n)
T(n) = T(n-k) + k
n-k = 0
n = k
T(n) = T(n-n) + n
= 0 + n
= O(n) → Time complexity
```

CONCLUSION:

The factorial program demonstrates both iterative and recursive approaches. Iterative factorial runs in linear time $O(n)$ with constant space, making it efficient. Recursive factorial also has $O(n)$ time but requires $O(n)$ stack space due to repeated function calls. Thus, iteration is more memory-efficient, while recursion is conceptually elegant.